

React Lab: Consuming REST APIs with `fetch()` and `componentDidMount()`

Hands-on Lab Solution — "fetchuserapp" (GetUser Component)

Objectives

- Explain how to consume REST APIs from React applications

Lab Goals

- Construct a React application that invokes a REST API and fetches data from it

Prerequisites

- Node.js
- NPM
- Visual Studio Code

A Note on the API Endpoint

The current, correct base endpoint for the Random User Generator API is <https://randomuser.me/api/> (not <https://api.randomuser.me/>, which is not a valid host for this service). This lab uses the working endpoint, <https://randomuser.me/api/>, so the application runs correctly.

1. Consuming REST APIs from React Applications

A REST API exposes data over HTTP, typically returning JSON. A React component consumes such an API by making an HTTP request (most commonly with the built-in `fetch()` function, though libraries like `axios` are also common), waiting for the response, and then storing the resulting data in the component's state so it can be rendered. Because network requests are asynchronous — they take time and the component cannot simply wait for them — this pattern always involves three states of the UI over time: a temporary 'loading' state before the data arrives, an 'error' state if the request fails, and finally the 'loaded' state once data is available.

`fetch(url)` returns a Promise that resolves to a Response object once the network request completes. Calling `response.json()` returns a second Promise that resolves to the actual parsed JavaScript object once the response body has been read. Both steps are commonly chained with `.then()`, and any failures are caught with `.catch()`.

```
fetch('https://randomuser.me/api/')
  .then((response) => response.json())
  .then((data) => {
    // 'data' is now a plain JavaScript object built from the JSON response
  })
  .catch((error) => {
    // runs if the network request fails or the JSON can't be parsed
  });
```

The recommended place to start a data-fetching request in a class component is the `componentDidMount()` lifecycle method. It runs exactly once, right after the component has been rendered to the DOM for the first time, which makes it safe to trigger side effects like network calls there. Fetching data in `componentDidMount()` rather than in the constructor or in `render()` means: (1) the initial render can show a simple loading UI immediately, and (2) `render()` is called again automatically, with the new data, once `setState()` is called inside the

.then() callback — render() itself is never used to fetch anything directly, because it must stay a pure, synchronous function of state and props.

```
componentDidMount() {  
  fetch('https://randomuser.me/api/')  
    .then((response) => response.json())  
    .then((data) => {  
      this.setState({ ...data.extracted here..., loading: false });  
    });  
}
```

The Random User Generator API used in this lab returns a JSON object with a results array containing one user object. Each user object has a nested name field (with title and first) and a nested picture field (with a large image URL), which is exactly the shape this lab needs to display title, first name, and a photo.

```
{  
  "results": [  
    {  
      "gender": "female",  
      "name": { "title": "Miss", "first": "Jennie", "last": "Nichols" },  
      "picture": {  
        "large": "https://randomuser.me/api/portraits/women/74.jpg",  
        "medium": "https://randomuser.me/api/portraits/med/women/74.jpg",  
        "thumbnail": "https://randomuser.me/api/portraits/thumb/women/74.jpg"  
      }  
    }  
  ]  
}
```

Lab Steps

Step 1: Create the React application

```
npx create-react-app fetchuserapp  
cd fetchuserapp  
npm start
```

Open the fetchuserapp folder in Visual Studio Code. Inside the src folder, create a new file: GetUser.js.

Step 2: Create the GetUser component

GetUser is a class component. Its constructor initializes state for the title, firstName, picture, a loading flag, and an error slot. componentDidMount() calls fetch() against the Random User API, pulls the first (and only) user out of the results array, and stores the three needed fields in state. render() then displays a loading message, an error message, or the user's title, first name, and photo, depending on the current state.

src/GetUser.js

```
import React, { Component } from 'react';  
  
class GetUser extends Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      title: '',  
      firstName: '',
```

```

        picture: '',
        loading: true,
        error: null
    };
}

componentDidMount() {
    fetch('https://randomuser.me/api/')
        .then((response) => {
            if (!response.ok) {
                throw new Error('Network response was not ok');
            }
            return response.json();
        })
        .then((data) => {
            const user = data.results[0];
            this.setState({
                title: user.name.title,
                firstName: user.name.first,
                picture: user.picture.large,
                loading: false
            });
        })
        .catch((error) => {
            this.setState({ error: error.message, loading: false });
        });
}

render() {
    const { title, firstName, picture, loading, error } = this.state;

    if (loading) {
        return <p>Loading user...</p>;
    }

    if (error) {
        return <p>Something went wrong: {error}</p>;
    }

    return (
        <div style={{ textAlign: 'center', fontFamily: 'Arial, sans-serif' }}>
            <h2>Random User</h2>
            <img
                src={picture}
                alt={firstName}
                style={{ borderRadius: '50%', width: '150px', height: '150px' }}
            />
            <p>{title} {firstName}</p>
        </div>
    );
}
}

export default GetUser;

```

Step 3: Update App.js

src/App.js

```
import React from 'react';
import GetUser from './GetUser';

function App() {
  return (
    <div className="App">
      <GetUser />
    </div>
  );
}

export default App;
```

Step 4: Render with ReactDOM

src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Step 5: Run the application

```
npm start
```

This opens the app at <http://localhost:3000>. On first load, “Loading user...” appears briefly while `componentDidMount()` fetches the data; once the promise resolves, the component re-renders with the fetched title, first name, and photo. Refreshing the page fetches a new random user each time, since the API returns a different person on every call.

Expected Output

```
Random User

(circular profile photo)

Miss Jennie
```

Summary

- A REST API is consumed from React using `fetch(url)`, which returns a Promise resolving to a Response, whose `.json()` method returns another Promise resolving to the parsed data.
- `componentDidMount()` is the standard place to start a data fetch in a class component, because it runs once right after the first render, keeping `render()` itself synchronous and side-effect free.
- While the request is in flight, the component shows a loading state; once `setState()` runs inside the `.then()` callback, React automatically re-renders with the fetched data.
- Errors from the network request or JSON parsing are caught with `.catch()` and surfaced through an error state, rather than left unhandled.

- In this lab, GetUser extracts title, first name, and photo URL from the nested name and picture fields of the Random User API's response and displays them once loading completes.